

Informatika pre biológov

Broňa Brejová

24.9.2026

Formulácia problému, algoritmus

- **Formulácia problému:** jasne zadefinujeme vstupné a výstupné dáta a aký výstup očakávame pre každý vstup.
- Vo formulácii nehovoríme **akým spôsobom** vypočítame výstupy zo vstupov.
- **Správny algoritmus:** Postup, ktorý určuje spôsob, akým pre každý vstup vypočítame príslušný výstup.

Príklad formulácie problému

Biologický problém: Pomocou hmotnostného spektrometra (mass spectrometer) sme odmerali vo vzorke peptid s hmotnosťou K . Máme databázu proteínov a chceme zistiť, ktorý z proteínov obsahuje peptid s touto hmotnosťou.

Informatický problém: Vstup je postupnosť n kladných čísel $a[1], \dots, a[n]$ a číslo K . Nájdite súvislý úsek tejto postupnosti $a[i], a[i + 1], \dots, a[j]$, ktorý má súčet K .

Príklad:

$K=19$

3 4 6 3 6 4 9 2 8

^^^ ^^ ^^

Ako riešiť tento problém?

Informatický problém: Vstup je postupnosť n kladných čísel $a[1], \dots, a[n]$ a číslo K . Nájdite súvislý úsek tejto postupnosti $a[i], a[i + 1], \dots, a[j]$, ktorý má súčet K .

Jednoduchý algoritmus: skúšame všetky možnosti

```
K = 19
a = [3, 4, 6, 3, 6, 4, 9, 2, 8]
for i in range(len(a)):          # všetky i od 0 po n-1
    for j in range(i, len(a)):  # všetky j od i po n-1
        suma = 0;
        for u in range(i, j+1): # všetky u od i po j vrátane
            suma = suma + a[u]
        if suma == K:
            print(i, j)
```

Ako dlho takýto program pobeží?

- Naimplementovať, spustiť a odmerať čas
- Na akom počítači? Na akých vstupoch?
- **Časová zložitosť**: počet operácií, ktoré program vykoná, v závislosti od množstva dát.
 - Zvolíme si parameter charakterizujúci množstvo dát
napr. počet prvkov vstupnej postupnosti n
 - Pre každú veľkosť vstupu odhadneme najhorší prípad
 - Robíme len hrubý odhad, zanedbáme (niektoré) konštanty

Časová zložitosť nášho programu

```
for i in range(len(a)):          # všetky i od 0 po n-1
    for j in range(i, len(a)):  # všetky j od i po n-1
        suma = 0;
        for u in range(i, j+1): # všetky u od i po j vrátane
            suma = suma + a[u]   # ***
        if suma == K:
            print(i, j)
```

Sčítavanie označené hviezdíčkami sa vykoná $n^3/6 + n^2/2 + n/3$ krát

Pre veľké n môžeme zanedbať $n^2/2 + n/3$, lebo $n^3/6$ je oveľa väčšie

Zanedbáme teda menej rýchlo rastúce členy a konštantu $1/6$

Časová zložitosť: kubická, alebo $O(n^3)$

Prečo je časová zložitosť dôležitá a konštanty nie?

		$O(n)$	$O(n^2)$	$O(n^3)$	$O(2^n)$
Čas na	20	<0.1s	<0.1s	<0.1s	<0.1s
vyriešenie	30	<0.1s	<0.1s	<0.1s	4s
problému	50	<0.1s	<0.1s	<0.1s	48 dní
veľkosti	100	<0.1s	<0.1s	<0.1s	4 mil. × vek vesmíru
	1 000	<0.1s	<0.1s	2s	...
	10 000	<0.1s	0.2s	0.5 hod	...
	100 000	<0.1s	20s	23 dní	...
	1 mil.	<0.1s	0.5 hod	63 rokov	...
	10 mil.	<0.1s	2 dni	63 tis. rokov	...
	100 mil.	0.2s	231 dní	63 mil. rokov	...
	10^9	2s	63 rokov	...	...
Zvýšenie	+1	—	—	—	×2
času so	×2	×2	×4	×8	—
zvýšeným n	×10	×10	×100	×1000	—

Efektívnejší algoritmus

- Najprv si predpočítame pre každý začiatok postupnosti jej súčet

$S[i] = a[1] + a[2] + \dots + a[i]$

a:	3	4	6	3	6	4	9	2	8
S:	3	7	13	16	22	26	35	...	

```
S[0] = a[0]
for i in range(1, len(a)):
    S[i] = S[i-1] + a[i]
```

- Potom súčet podpostupnosti od i po j vieme spočítať jednoducho ako

$S[j] - S[i - 1]$

```
for i in range(len(a)):          # všetky i od 0 po n-1
    for j in range(i, len(a)):    # všetky j od i po n-1
        suma = S[j]-S[i-1]        # treba ešte opraviť i=0
        if suma == K:
            print(i, j)
```

- **Časová zložitosť:** kvadratická, alebo $O(n^2)$
- Ak sú všetky čísla kladné, dá sa aj v lineárnom čase $O(n)$

Ďalší příklad informatického problému z prednášky

Najkratšie spoločné nadslovo

- Vstup: niekoľko reťazcov
- Výstup: najkratší reťazec, ktorý obsahuje všetky vstupné reťazce ako súvislé podreťazce

Príklad:

Vstup: GCCAAC, CCTGCC, ACCTTC

Výstup: CCTGCCAACCTTC (najkratšie možné)

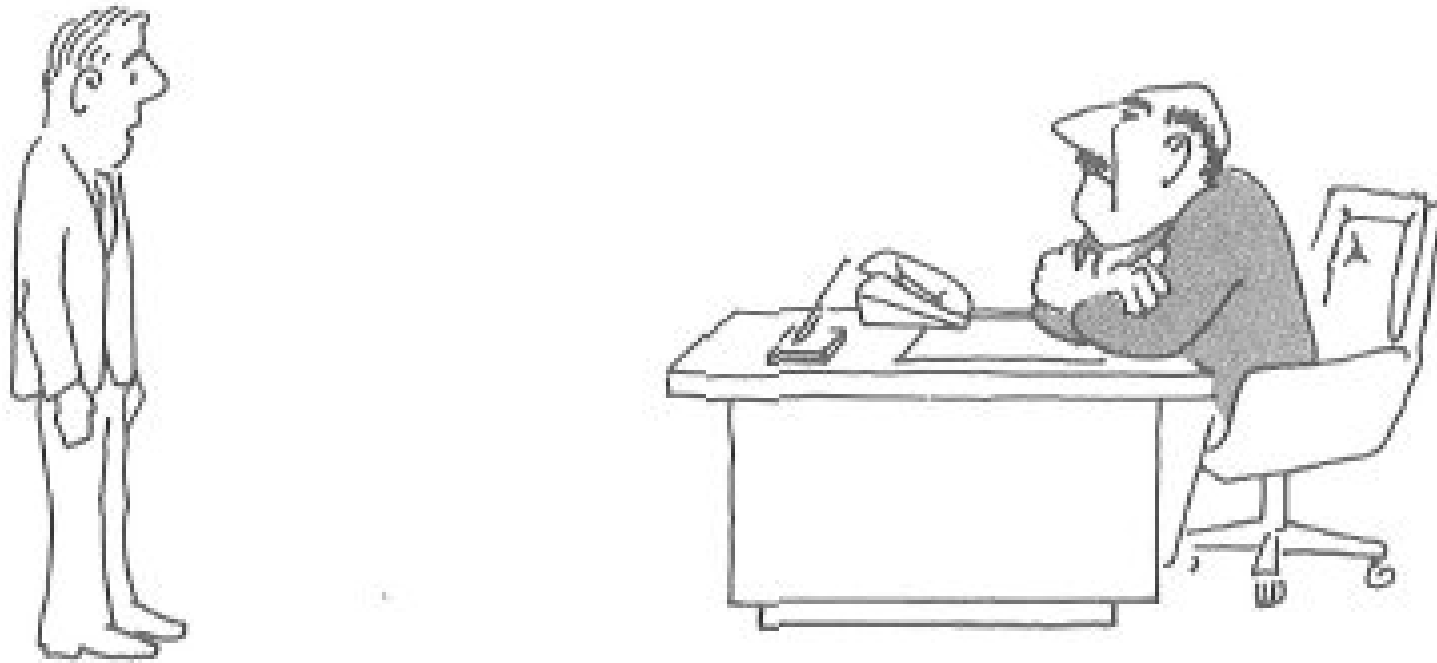
Ako rýchly algoritmus poznáme pre tento problém?

Najkratšie spoločné nadslovo

Nepoznáme algoritmus, ktorý by bežal v polynomiálnom čase

t.j. $O(n^k)$ pre nejakú konštantu k

Tento problém je **NP-t'ažký**.



“I can’t find an efficient algorithm, I guess I’m just too dumb.”



“I can’t find an efficient algorithm, because no such algorithm is possible!”



"I can't find an efficient algorithm, but neither can all these famous people."

Ako sa vysporiadať s NP-ťažkými problémami?

Heuristické algoritmy

- Nájde **aspoň nejaké riešenie**, aj keď nie nutne optimálne
- Nejde teda o správny algoritmus riešiaci náš problém, lebo pre niektoré vstupy dáva zlú odpoveď
- Radšej ale horšia odpoveď rýchlo, ako perfektná o milión rokov

Príklad: Heuristika pre najkratší spoločný nadreťazec: v každom kroku zlepíme dva reťazce s najväčším prekryvom

Príklad: CATATAT, TATATA, ATATATC

Optimum: CATATATATC, dĺžka 10

Heuristika: CATATATCTATATA, dĺžka 14

Ako so vysporiadať s NP-tažkými problémami?

Aproximačný algoritmus

- Často vieme dokázať, že nejaká heuristika sa vždy priblíži k optimálnemu riešeniu aspoň po určitú hranicu

Príklad: Heuristika pre najkratší spoločný nadreťazec: v každom kroku zlepíme dva reťazce s najväčším prekryvom

Je dokázané, že vždy nájde najviac 3,5-krát dlhší reťazec ako najlepšie riešenie.

Informatici predpokladajú, že v skutočnosti najviac 2-krát dlhší, ale nevieme to dokázať.

Ako so vysporiadať s NP-tažkými problémami?

Exaktný výpočet pomocou iného problému

- Preformulovať do podoby jedného z dobre známych NP-tažkých problémov (napr. celočíselné lineárne programovanie)
- Múdri ľudia napísali programy, ktoré vedia riešiť tieto známe problémy **aspoň v niektorých prípadoch** (napr. Gurobi, CPLEX)

Preformulovať problém

- Je toto skutočne jediná rozumná formulácia biologického problému ktorý chceme vyriešiť?

Zhrnutie

- Problémy zo skutočného života je dobré najskôr sformulovať tak, aby bolo jasné, aké výsledky očakávame pre každý možný vstup.
- Takáto formulácia by mala byť oddelená od postupu (algoritmu) riešenia.
- Informatici merajú čas v O-čkach, ktoré abstrahujú od detailov konkrétneho počítača.
- Vytvorenie efektívneho algoritmu je umenie. Časť z toho sú finty (ako napr. dynamické programovanie).
- Pre niektoré problémy poznáme iba Nechutne Pomalé algoritmy (NP-ťažké).
- Aj napriek tomu vo veľa prípadoch vieme pomôcť.